API Gateway Pattern

Category: Edge / routing **One-liner:** A single entry point that sits in front of all microservices, routing, aggregating, securing, and shaping requests so clients never talk to services directly.

Lesson 1 — Introduction to the API Gateway Pattern

The fast-food counter

Picture ordering a burger. You walk to the counter, place your order, pay, and wait. That counter is the API Gateway. It is the **single point at which you, the customer, interact with the restaurant's services** — the kitchen, the billing system, inventory. You never walk into the kitchen to ask the cook for your patty and then go to a separate desk to pay. One counter, one interaction, and everything behind it is somebody else's problem.

What the pattern actually is

In a **microservice architecture** (an application split into many small services, each running separately and owning one business capability) you have many services each doing their own thing. The API Gateway Pattern is about having **a single entry point for external consumers** to reach all of them.

Three things follow from that single entry point, and it's worth separating them because they're independent benefits:

1. **It makes the system easier to manage** — there is one place where request handling policy lives.
2. **It secures your services** — the services themselves need not be individually exposed.
3. **It provides a unified interface to the external world** — clients see one coherent API rather than the seams between your teams.

Why the pattern exists

The dominant architectural trend of recent years has been the shift from **monolithic** structures (one big application built and deployed as a single unit) to microservices. The course compares it to the shift from landlines to smartphones in telecommunications — a genuine step change that brought enhanced **scalability** (the ability to handle more work by adding resources), increased flexibility, and more. But as with any technological advance, it introduced its own problems.

The biggest of those problems is communication:

- **How do you ensure effective and efficient communication** between a large number of services?
- **How do you manage the growing complexity** that comes with an ever-increasing number of interactions between them?

This is precisely the gap the API Gateway Pattern fills.

The Netflix example

When you open Netflix on your TV or phone, the app needs a lot of different data — the movie list, your user preferences, subtitles, and more. Netflix has a separate microservice responsible for each of those pieces.

Now consider the two possible designs:

- **Without a gateway:** your TV app calls each microservice separately. It must know that there *is* a preferences service, where it lives, how to **authenticate** to it (prove who you are, e.g. with a

password or token), and what shape its response takes — and repeat all of that for every other service.

- **With a gateway:** your TV app makes one call to the Netflix API Gateway, which then talks to whichever microservices are needed.

The second is dramatically simpler for the TV app. And note *which* app benefits most — a TV app or a mobile app is something you cannot force users to update. Every piece of **service topology** (which services exist, where they run and how they connect) you bake into it is frozen in the field for years.

The master orchestrator

The gateway operates like **the conductor of an orchestra**. It oversees and directs the entire ensemble of services, ensuring each plays its part at the right time to produce a harmonious output.

Mechanically, it does three things in sequence:

1. Serves as the **single entry point** for all client requests
2. **Routes** those requests to the appropriate microservices
3. **Gathers the responses** from the services involved and delivers a **unified response** back to the client

The beauty of the pattern is its ability to **mask the underlying complexity from clients**. As a client you do not need to know how many services were involved or how they communicate with each other. You make a request; the gateway handles the rest.

The headline advantages

The introduction gives four, which the next lesson expands considerably:

- **Simplified client** — clients interact with one gateway instead of many services.
- **Security** — **authentication** (checking who the caller is) and **authorization** (checking what that caller is allowed to do) can be added at the gateway level.
- **Performance** — you can add **caching** (keeping copies of recent answers so repeat requests are served without redoing the work), compress responses (shrink them so they travel faster), and manage **load balancing** (spreading requests across several copies of a service). See System Design Fundamentals for both.
- **Cross-cutting concerns** (needs that every service shares, rather than belonging to one feature) — **logging** (writing a record of what happened), **analytics** (collecting usage data), and similar concerns are handled in one place.

Lesson 2 — Advantages: the fifteen problems it solves

The framing

Imagine a city full of different services and places to visit. As a tourist, interacting with each service individually gets confusing and overwhelming fast. But if there is a **single information centre** that guides you and helps you interact with all of them, your life becomes far easier. That is what the gateway does for microservices.

What follows is each problem the course identifies, with the gateway's answer. I've kept all fifteen as their own item because each is a distinct argument.

1. Complexity of interactions

Problem: In a microservices architecture there are many services talking to each other. If every service communicates with all the others directly, the number of connections grows very complex,

very fast. **Solution:** The gateway becomes the central point of communication. Services talk to the gateway, and the gateway talks to other services — collapsing a mesh of connections into a hub.

2. Inconsistency in service calls

Problem: Different services may require different interaction styles and have different **data formats** (ways of encoding data, e.g. JSON, XML or binary). A client ends up implementing several dialects of your own API. **Solution:** The gateway provides one consistent way to interact with everything. It can **translate calls and data formats**, so the client sees uniformity even where the **backend** (the server-side services behind the gateway) has none.

3. Security and authentication

Problem: Handling security for each service individually is a nightmare. Every service must be independently secured, and that is a great deal of duplicated, easily-inconsistent work. **Solution:** Authentication and authorization are handled **centrally**. Clients authenticate with the gateway, and the gateway ensures callers are who they claim to be before anything reaches a service.

4. Load balancing

Problem: Some services receive far more traffic than others, and that traffic needs managing efficiently. **Solution:** The gateway distributes incoming traffic evenly across services, ensuring **no single service is overwhelmed**.

5. Performance issues

Problem: Services are spread out, and the network calls between them add delays. **Solution:** The gateway can **cache responses** from services, reducing the need for repeated calls and improving performance.

6. Cross-cutting concerns

Problem: Concerns like logging, monitoring, and analytics would otherwise need implementing in **every single service**. **Solution:** Handle them centrally at the gateway, removing that duplication.

7. Managing legacy systems

Problem: You may have old **legacy** services (old systems still running because the business depends on them) that must be accessed in specific, awkward ways. **Solution:** The gateway can interact with legacy services while **presenting a modern interface** to clients — the awkwardness stops at the gateway boundary.

8. Version management

Problem: Different clients may need to interact with different versions of a service — again, most acute with mobile and TV clients you cannot force to upgrade. **Solution:** The gateway routes requests to the appropriate version of a service based on the client's needs.

9. Aggregation of results

Problem: A client may need information spread across multiple services. Gathering it could require several network calls, which is inefficient — especially over a mobile connection. **Solution:** The gateway aggregates results from multiple services and returns a **single consolidated response**.

10. Service orchestration

Problem: Some operations require a series of service calls made **in a particular order**. **Solution:** The gateway orchestrates those calls, ensures correct ordering, and handles any transformations needed along the way.

11. Handling partial failures

Problem: In a **distributed system** (one made of many programs on different machines that cooperate over a network) some services fail while others keep working. Handling these **partial** failures gracefully is genuinely difficult. **Solution:** The gateway detects failures and either **reroutes** requests or provides **fallback responses** (a safe default answer used when the real one can't be obtained), so the system stays robust rather than failing wholesale.

12. Request transformation

Problem: Different services expect requests formatted in different ways. **Solution:** The gateway transforms each incoming request into the format the target service expects.

13. Client-specific adaptations

Problem: Different clients — mobile, web, and so on — have different requirements. **Solution:** The gateway adapts responses to suit each client's needs. *(Pushed to its logical conclusion, this becomes BFF — one gateway per client type.)*

14. Centralized logging and monitoring

Problem: Collecting logs and monitoring data from every service in a distributed system is challenging. **Solution:** The gateway collects logs and monitoring data, giving you **one point for observability** (the ability to understand what a running system is doing from its logs, metrics and traces).

15. Handling asynchronous operations

Problem: Some operations take a long time and should not block the client. **Solution:** The gateway manages those **asynchronous** operations (work that continues in the background while the caller moves on), giving clients a way to **check progress or receive a callback** on completion (a **callback** is a call made back to the client when the work finishes).

Taken together, these create a smoother, more reliable interaction layer for clients — the course describes it as a super-efficient helper that takes care of the nitty-gritty details so you get a seamless experience.

Lesson 3 — Worked example: a library system

The scenario

A library system in which various clients — library users, staff, and management — interact with several microservices: a **Catalogue Service**, a **User Service**, and an **Inventory Service**.

Each service corresponds to a distinct area of the library's functionality:

- **CatalogueService** handles requests relating to book searches
- **UserService** manages user profiles
- **InventoryService** keeps track of book stock levels

In the Java implementation each would be an **interface** (a Java type that lists method signatures without implementing them) with methods matching potential client requests — `CatalogueService`, for instance, might expose `searchByTitle(String title)` and `searchByAuthor(String author)`.

The gateway

A separate class acting as the single point of contact for all client requests. The course's analogy: **a receptionist at the library**, fielding questions from visitors and directing them to the right place.

The crucial design point: `ApiGateway` has a method corresponding to each client request, mirroring the services — but rather than handling those requests itself, **each method delegates the work to the appropriate service**.

```
public class ApiGateway {
    private UserService userService;
    private CatalogueService catalogueService;
    private InventoryService inventoryService;

    public ApiGateway(UserService userService,
                      CatalogueService catalogueService,
                      InventoryService inventoryService) {
        this.userService = userService;
        this.catalogueService = catalogueService;
        this.inventoryService = inventoryService;
    }

    public User getUserDetails(String userId) {
        return userService.getUserDetails(userId);
    }

    public Book searchByTitle(String title) {
        return catalogueService.searchByTitle(title);
    }

    // ...additional methods as needed...
}
```

Walking it through: `ApiGateway` receives a request for user details and passes it to `UserService`. The same applies to a book title search, which is routed to `CatalogueService`.

What to take from the shape, not the detail: gateway methods *delegate*, they never *implement*. The moment a gateway method starts containing **business rules** (logic about the domain itself, e.g. "members may borrow 5 books") rather than forwarding, you are on the road to the **distributed monolith** described below (a system split into services that are still so tangled they must change and deploy together). Real gateways would wrap each delegation with auth, **timeouts** (a maximum wait before giving up), **retries** (trying a failed call again), and caching — but the delegation shape stays the same.

Lesson 4 — Problems the gateway itself introduces

The course opens this lesson with "every rose has its thorn" — the gateway solves a great deal and becomes a liability of its own. Five problems, each with its mitigation.

1. Single point of failure

Problem: If the API Gateway goes down, **nothing can communicate**. You have concentrated all traffic through one component, so its availability is now your system's availability. **Solution:** Implement **high availability** (staying up nearly all the time) and **redundancy** (running spare copies so one failure isn't fatal) for the gateway, and **monitor it diligently**.

2. Potential bottleneck

(A **bottleneck** is the one narrow point that limits how fast everything else can go.) **Problem:** All requests pass through the gateway, which could cause delays. **Solution:** Ensure the gateway is highly performant, **scale it horizontally** (add more machines running copies, rather than a bigger machine), and optimise its code.

3. Complexity in the gateway

Problem: The gateway can become too complex as it accumulates routing, security, transformations, and everything else. **Solution:** Keep the gateway's responsibilities clear. **Consider splitting it into multiple gateways** if necessary — per client type, or per functionality.

> This is the failure mode worth naming explicitly: the gateway is where business logic goes to hide. Once teams start putting domain rules in it, every team needs to change it, and it becomes the deployment bottleneck microservices were supposed to eliminate. Keep it about **transport** (moving requests to the right place), not **domain** (the business problem the software models).

4. Maintenance overhead

Problem: As services evolve, maintaining the gateway's routes and rules becomes challenging — the gateway's configuration drifts away from reality. **Solution: Automate as much as possible.** Use **CI/CD** (Continuous Integration / Continuous Delivery — automatically building, testing and releasing every code change) to ease maintenance.

5. Security risk

Problem: Being the entry point makes it a **prime target for attacks**. The same concentration that makes it useful makes it valuable to an attacker. **Solution:** Implement robust security practices, keep the gateway updated, and monitor for **vulnerabilities** (weaknesses an attacker could exploit).

Lesson 4 (cont.) — Performance implications

Distinct from the problems above: these are the costs the gateway imposes on every request.

1. Latency

Implication: The gateway introduces an **additional network hop** (one more trip between machines), which adds **latency** — the time a request takes from start to finish. **Solution:** Optimise its performance using techniques like **caching and response compression**.

2. Resource utilization

Implication: The gateway may consume significant resources, affecting performance. **Solution:** Ensure it has adequate resources; monitor utilisation and **scale as necessary**.

3. Content aggregation overhead

Implication: Aggregating responses from multiple services at the gateway is resource-intensive. **Solution:** Optimise the aggregation logic and use **asynchronous operations** where possible to prevent blocking.

4. Rate limiting challenges

Implication: Implementing **rate limiting** (capping how many requests a client may make per time window) at the gateway is complex, **especially in distributed deployments** — several gateway **instances** (running copies of the same program) must share a view of each client's usage. **Solution:** Use advanced algorithms built for this, such as **token bucket** or **leaky bucket**, which handle distributed scenarios.

> **Key term — token bucket / leaky bucket:** In a **token bucket**, each client has a bucket that refills with tokens at a steady rate; each request spends one token, and when the bucket is empty

> requests are rejected. A full bucket allows a short burst. A **leaky bucket** instead queues requests and lets them out at a fixed rate, smoothing bursts away. Both are explained further in System Design Fundamentals.

5. Load balancing overhead

Implication: The gateway must distribute requests efficiently to prevent **hotspots** (single instances that get far more traffic than the rest). **Solution:** Implement smart load balancing algorithms; **consider a service mesh** (a layer of small proxies beside every service that handles service-to-service traffic) for more granular control. *(See Sidecar.)*

6. Handling asynchronous operations

Implication: Managing async operations at the gateway is complex and can affect performance. **Solution:** Optimise their handling and provide clients with **clear mechanisms to check status or receive callbacks**.

The overall verdict: the pattern introduces challenges and performance costs, but with careful design, monitoring, and optimisation you can mitigate them and run a smooth, efficient system. The alternative — every client knowing every service — is worse.

Lesson 5 — System design example: Instagram

Instagram is a good illustration because it is genuinely complex: millions of users and a great many features. Think of it as a **bustling city**, with the API Gateway as the **main entrance**, directing traffic and keeping everything running smoothly.

The gateway's responsibilities here

- **Routing** — directing user requests to the appropriate microservice
- **Aggregation** — combining data from multiple services into one consolidated client response
- **Authentication & authorization** — confirming users are who they say they are, and that they have permission to access what they requested
- **Rate limiting** — preventing abuse by limiting how many requests a user can make in a timeframe
- **Load balancing** — distributing requests across multiple instances of a service so none is overwhelmed
- **Logging and monitoring** — tracking what is happening for troubleshooting and performance monitoring

The microservices behind it

- **User Management Service** — user profiles, relationships (followers/following), account settings
- **Photo Upload and Processing Service** — uploading photos, applying filters, resizing for different devices
- **Feed Service** — generating and serving the user's feed of photos from people they follow
- **Notification Service** — notifications for new followers, likes, comments
- **Search Service** — queries for user profiles, hashtags, and locations
- **Comment Service** — comments on photos

Scenario A — uploading a photo *(sequential)*

1. The user uploads a photo via the Instagram app
2. The request hits the API Gateway
3. The gateway **authenticates** the user and checks they are **authorized to upload photos**

4. Once authenticated, the gateway routes the request to the Photo Upload and Processing Service
5. The Photo Service processes the image and sends a response back to the gateway
6. The gateway sends a confirmation back to the user's app

Note that authentication and authorization are **two separate checks** at step 3, and both happen before any service is touched.

Scenario B — viewing the feed *(parallel — the interesting one)*

1. The user opens the app to view their feed
2. The request hits the API Gateway
3. The gateway authenticates the user
4. The gateway makes **parallel requests** to:
    - **Feed Service** — to get the list of recent photos from people the user follows
    - **Notification Service** — to fetch any new notifications
5. The gateway **aggregates** the responses from these services
6. The gateway sends a **single consolidated response** back to the app, showing feed and notifications together

Why the contrast matters

Upload is a single routed call. Feed is a **scatter-gather**: one client request fans out to several parallel service calls and comes back as one merged response.

That fan-out is where the gateway earns its latency cost. Doing the same thing client-side would mean several sequential **round trips** (a request going out and its response coming back) over a mobile network — far slower than one round trip plus parallel calls inside the **data centre** (the building of servers where the services run), where latency between services is a fraction of the latency to the phone.

The result: Instagram gets a consistent, efficient, and secure way to handle all the interactions needed for a seamless experience across millions of users, and the system stays easier to manage and scale as it grows.

Consolidation

API Gateway vs. BFF

	API Gateway	BFF
Count	One shared entry point	One per client type
Optimised for	All clients generically	One specific client
Client-specific logic	Pollutes it	Belongs there
Risk	Becomes a bottleneck / distributed monolith	Duplication and drift across BFFs

Not mutually exclusive — a common setup is a thin shared gateway (TLS — the encryption behind HTTPS; coarse auth; rate limiting) with BFFs behind it.

When to use

- Multiple microservices **and** multiple client types
- Clients would otherwise make many round trips per screen
- You want cross-cutting concerns (auth, rate limiting, logging) in one place

- You are exposing services to untrusted external consumers

When to avoid

- Very few services — the gateway is pure overhead
- Purely internal service-to-service traffic — a **service mesh** fits better
- One client whose needs diverge sharply from everything else — consider BFF
- Ultra-low-latency paths where an extra hop is unacceptable

Interview soundbite (30 seconds)

> "An API Gateway is a single entry point in front of your microservices. Clients call it instead of calling services directly, and it handles routing, response aggregation, authentication, rate limiting, caching, and logging in one place. The big win is that clients don't need to know your service topology — which matters most for mobile and TV clients you can't force-update, because any topology you bake into them is frozen in the field for years. The classic example is Instagram's feed: one request fans out in parallel to the feed and notification services and comes back as one response, which beats several sequential round trips from the phone. The trade-offs are that it's a single point of failure needing HA (high availability) and redundancy, it adds a network hop, rate limiting across distributed gateway instances needs token or leaky bucket algorithms, and it tends to accumulate business logic until it becomes a distributed monolith — so keep it about transport, not domain, and split it per client type or per domain if it gets too big. If each client type needs a genuinely different API shape, that's Backends for Frontends."

Implementation

A. Runnable demo

Full source: `demos/ApiGatewayDemo.java`

```
cd demos && ./gradlew runCh03
```

The heart of it is scatter-gather with per-service fallbacks:

```
Future<String> fFeed    = pool.submit(() -> feed.fetch(userId));
Future<String> fNotifs  = pool.submit(() -> notifications.fetch(userId));
Future<String> fProfile = pool.submit(() -> profile.fetch(userId));

result.put("feed",          get(fFeed,    "feed-unavailable"));
result.put("notifications", get(fNotifs,  "no-notifications"));
result.put("profile",       get(fProfile, "profile-unavailable"));

private String get(Future<String> f, String fallback) {
    try { return f.get(1, TimeUnit.SECONDS); }
    catch (Exception e) { return fallback; }        // partial failure contained
}
```

Here `pool` is a **thread pool** (a set of reusable worker threads that run tasks), and each `Future` is a placeholder for a result that will arrive later. All three calls are **submitted before any result is read** — that's what makes them parallel. Reading `fFeed` before submitting the others would **serialise** them (force them to run one after another).

B. What you should see

```
=== 1. Without a gateway: client calls each service itself ===
  3 round trips from the client: 288ms
  >> on mobile add ~150ms latency PER trip: ~738ms

=== 2. Gateway, sequential aggregation ===
  1 client round trip, 3 sequential server calls: 279ms

=== 3. Gateway, PARALLEL scatter-gather ===
  1 client round trip, 3 parallel server calls: 129ms
  >> ~the slowest service (120ms), not the sum

=== 4. Auth enforced once, at the edge ===
  rejected: 401 Unauthorized  (no service was touched)

=== 5. Partial failure -> graceful degradation ===
  {feed=feed-data, notifications=no-notifications, profile=profile-data}

=== 6. Rate limiting, also at the edge ===
  429 Too Many Requests after 5 requests
```

(The numbers `401` and `429` are **HTTP status codes** — standard three-digit codes in every web response; 401 means "not authenticated", 429 means "too many requests".)

Compare 2 and 3: 279ms → 129ms. Parallel aggregation costs the *slowest* service, not the sum. And note **section 4** — an unauthenticated request touched **zero** backend services, which is the security argument for a gateway stated as an observable fact.

C. Production — Spring Cloud Gateway

Spring Cloud Gateway is a ready-made gateway built on **Spring**, the most widely used Java framework. The XML below is the **Maven dependency** that adds it to a project (Maven is a Java build tool that downloads libraries). In the YAML config, `lb://` means "look the service up via **service discovery**" (a registry of where each service is running — see Service Discovery), **predicates** decide which requests a route matches, and **filters** modify or guard the request on its way through. `GET` is the HTTP method for reading data; a request is **idempotent** if repeating it has the same effect as doing it once.

```
<dependency>
  <groupId>org.springframework.cloud</groupId>
  <artifactId>spring-cloud-starter-gateway</artifactId>
</dependency>
```

```yaml
spring:
  cloud:
    gateway:
      routes:
        - id: orders
          uri: lb://order-service          # lb:// = resolve via service discovery
          predicates:
            - Path=/api/orders/**
          filters:
            - StripPrefix=2
            - name: CircuitBreaker
              args:
                name: orderCB
                fallbackUri: forward:/fallback/orders
            - name: RequestRateLimiter
              args:
                redis-rate-limiter.replenishRate: 100    # steady-state req/s
                redis-rate-limiter.burstCapacity: 200    # burst allowance
            - name: Retry
              args:
                retries: 2
                statuses: BAD_GATEWAY,SERVICE_UNAVAILABLE
                methods: GET                             # only idempotent methods

        - id: products
          uri: lb://product-service
          predicates:
            - Path=/api/products/**
            - Header=X-API-Version, v2                   # route by version
```

Global cross-cutting filter — auth applied to every route. (`Mono` is a **reactive** type: a promise of one value delivered later without blocking a thread.)

```java
@Component
public class AuthFilter implements GlobalFilter, Ordered {
    @Override
    public Mono<Void> filter(ServerWebExchange exchange, GatewayFilterChain chain) {
        String token = exchange.getRequest().getHeaders().getFirst("Authorization");
        if (!tokenService.isValid(token)) {
            exchange.getResponse().setStatusCode(HttpStatus.UNAUTHORIZED);
            return exchange.getResponse().setComplete();      // never reaches a service
        }
        return chain.filter(exchange);
    }
    @Override public int getOrder() { return -100; }          // run early
}
```

Kubernetes-native alternative — (**Kubernetes** is a platform that runs and manages **containers** — packaged, isolated applications — across a cluster of machines.) An **Ingress** or **Gateway API** resource (Kubernetes objects describing how outside traffic enters the cluster) does routing, TLS, and rate limiting without you writing or operating a gateway application at all. Prefer this when your needs are routing-shaped rather than aggregation-shaped.

D. Implementation pitfalls

1. Sequential calls that should be parallel. The most common performance bug in gateways. If you `.get()` on each future as you create it, you've written sequential code that looks **concurrent** (several tasks in progress at the same time).

2. No per-call timeout in aggregation. One slow service makes the whole aggregated response slow. Always bound each future — `f.get(1, TimeUnit.SECONDS)` in the demo — so the slowest dependency can't dictate your latency.

3. Business logic in the gateway. The distributed-monolith trap. Once domain rules live there, every team must change it and it becomes the deployment bottleneck. Keep it to transport concerns: routing, auth, rate limiting, aggregation, observability.

4. In-memory rate limiting behind multiple gateway instances. Three **replicas** (identical running copies) each allowing 100 req/s (requests per second) means the real limit is 300. Use a **shared store (Redis)** — Redis is a very fast in-memory key-value database often used for counters and caches — which is why the config above uses `redis-rate-limiter`.

5. Retrying non-idempotent requests. The `methods: GET` line above is deliberate. A gateway retrying a **POST** (the HTTP method for creating something) can create duplicate orders. *(See Retry.)*

6. Running the gateway as a single instance. It's a single point of failure by design — so run at least two replicas behind a **load balancer** (a component that spreads incoming requests across several servers), and treat its resource limits and monitoring as seriously as any critical service.

7. Losing trace context. The gateway is the ideal place to generate a **correlation ID** (a unique ID attached to a request and passed along to every service it touches, so its logs can be joined up) and propagate it **downstream** (to the services it calls). Skip it and you lose the ability to trace a request across services.

Related patterns

- **BFF** — one gateway per client type instead of one shared gateway
- **Circuit Breaker** — the gateway is a natural place to configure breaker rules per backend route
- **Service Discovery** — how the gateway finds service instances; the gateway is the classic server-side discovery router
- **Strangler Fig** — the migration facade is often literally an API Gateway
- **Sidecar** — gateway handles north-south (external, in and out of the system) traffic; a mesh of sidecars handles east-west (internal, service-to-service)